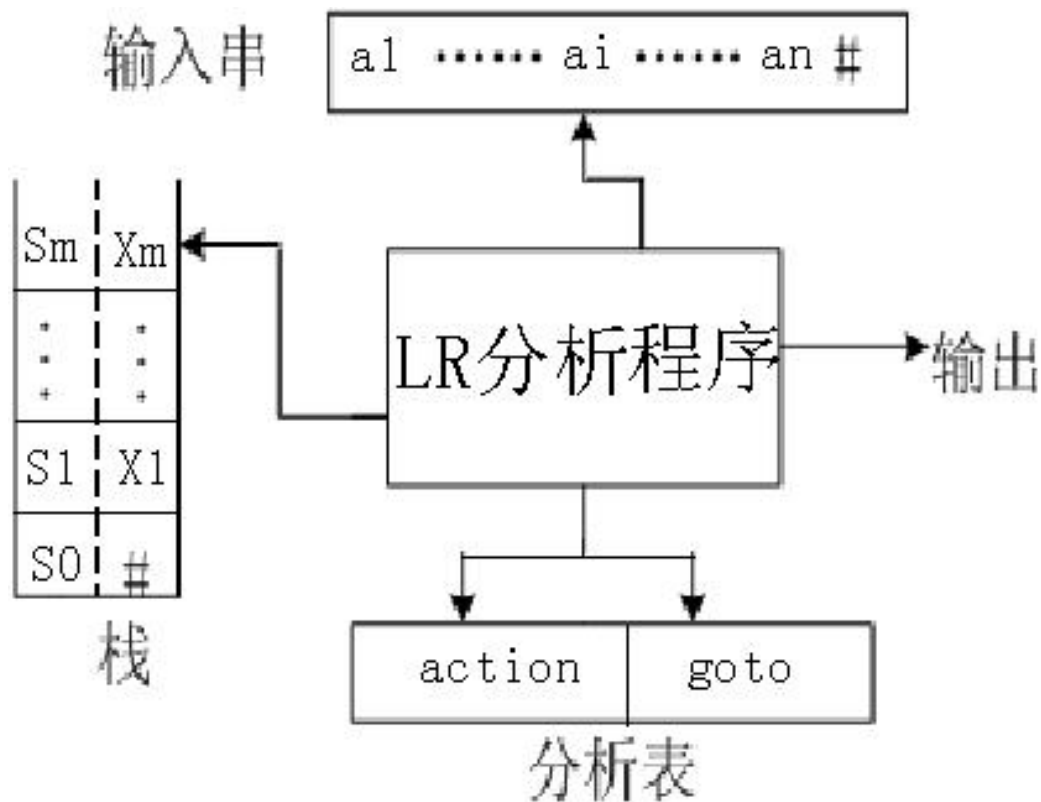


第三章--自下而上语法分析

自下而上语法分析方法的基本思想是移进—归约。归约是推导的逆过程。



回顾



分析器实际上是一个带有先进后出栈的确定的有穷自动机。分析栈用来存放状态，状态概括了从分析开始直到某一归约阶段的全部历史和展望资料，**根据栈顶状态和输入符号就可以唯一决定下一个动作。**

回顾

- **活前缀**：规范句型的一个前缀，不含句柄之后的任何符号。在它之后增添一些终结符号后，就成为规范句型。即：
- 对于文法G， 若 $S \Rightarrow a\beta$, $\beta \in V_t^*$ ，称a为**活前缀**。
- 在LR分析的过程中，假定输入串是一个句子，任何时候符号栈里的文法符号都构成活前缀，配上输入串的剩余部分，就成为规范句型。
- 构造一个有穷自动机来识别文法G的所有活前缀，这样就可以自动生成LR分析表。

回顾

- 写出文法的所有项目，每个项目是一个状态
- 规定项目1为NFA的唯一初态
- 若状态i和状态j出自同一产生式，而且状态j的圆点只落后于状态i一个位置：
 - 若i的圆点后是a，从i到j画一条弧，标记为a
 - 若i的圆点后是A，则连两种弧：(1)从状态i画 ϵ 弧到所有的 $A \rightarrow \cdot \beta$ 的状态。(2)从状态i到j画弧，标记为A
 - 归约项目表示结束状态，用双圈表示

3) LR(0)项目集规范族的构造:

- 一个项目集 I 的闭包 $\text{Closure}(I)$ 的计算:
 - (1) I 中的任何项目都 $\in \text{Closure}(I)$
 - (2) 若 $A \rightarrow \alpha \cdot B\beta \in \text{Closure}(I)$, 且 $B \in V_N$, 则对任何关于 B 的产生式: $B \rightarrow \cdot r \in \text{Closure}(I)$, r 为任意符号串
 - (3) 重复(2)直到 $\text{Closure}(I)$ 不再增加为止。

例：对于文法3.6有下列项目：

- | | | |
|------------------------------|-------------------------------|------------------------------|
| 1. $S' \rightarrow \cdot E$ | 2. $S' \rightarrow E \cdot$ | |
| 3. $E \rightarrow \cdot aA$ | 4. $E \rightarrow a \cdot A$ | 5. $E \rightarrow aA \cdot$ |
| 6. $A \rightarrow \cdot cA$ | 7. $A \rightarrow c \cdot A$ | 8. $A \rightarrow cA \cdot$ |
| 9. $A \rightarrow \cdot d$ | 10. $A \rightarrow d \cdot$ | |
| 11. $E \rightarrow \cdot bB$ | 12. $E \rightarrow b \cdot B$ | 13. $E \rightarrow bB \cdot$ |
| 14. $B \rightarrow \cdot cB$ | 15. $B \rightarrow c \cdot B$ | 16. $B \rightarrow cB \cdot$ |
| 17. $B \rightarrow \cdot d$ | 18. $B \rightarrow d \cdot$ | |

若项目集 $I = \{E \rightarrow a \cdot A\}$,

则 $\text{Closure}(I) = \{E \rightarrow a \cdot A, A \rightarrow \cdot cA, A \rightarrow \cdot d\}$

若项目集 $I = \{B \rightarrow c \cdot B\}$, 求 $\text{Closure}(I) = ?$

$\text{Closure}(I) = \{B \rightarrow c \cdot B, B \rightarrow \cdot cB, B \rightarrow \cdot d\}$

- 状态转换函数 $GO(I, X)$ 的计算:

$$GO(I, X) = \text{Closure}(J)$$

I 是一个项目集, X 是一个文法符号

其中 $J = \{\text{任何形如 } A \rightarrow \alpha X \cdot \beta \text{ 的项目} \mid A \rightarrow \alpha \cdot X\beta \in I\}$

- GO 函数实际就是检查 I 中的每一个后继符号为 X 的项目, 将这个圆点向后移动一个位置, 得到项目集 J , 再对项目集 J 求闭包。

例：对于上面的例题文法3.6有下列项

- | | | |
|------------------------------|-------------------------------|------------------------------|
| 1. $S' \rightarrow \cdot E$ | 2. $S' \rightarrow E \cdot$ | |
| 3. $E \rightarrow \cdot aA$ | 4. $E \rightarrow a \cdot A$ | 5. $E \rightarrow aA \cdot$ |
| 6. $A \rightarrow \cdot cA$ | 7. $A \rightarrow c \cdot A$ | 8. $A \rightarrow cA \cdot$ |
| 9. $A \rightarrow \cdot d$ | 10. $A \rightarrow d \cdot$ | |
| 11. $E \rightarrow \cdot bB$ | 12. $E \rightarrow b \cdot B$ | 13. $E \rightarrow bB \cdot$ |
| 14. $B \rightarrow \cdot cB$ | 15. $B \rightarrow c \cdot B$ | 16. $B \rightarrow cB \cdot$ |
| 17. $B \rightarrow \cdot d$ | 18. $B \rightarrow d \cdot$ | |

已知 $I = \{ E \rightarrow a \cdot A, A \rightarrow \cdot cA, A \rightarrow \cdot d \}$ 求 $go(I, A)$ 和 $go(I, c)$

$$go(I, A) = \{ E \rightarrow aA \cdot \}$$

$$go(I, c) = \{ A \rightarrow c \cdot A, A \rightarrow \cdot cA, A \rightarrow \cdot d \}$$

已知 $I = \{ B \rightarrow c \cdot B, B \rightarrow \cdot cB, B \rightarrow \cdot d \}$ 求 $go(I, B)$ 和 $go(I, c)$

$$go(I, B) = \{ B \rightarrow cB \cdot \}$$

$$go(I, c) = \{ E \rightarrow c \cdot B, B \rightarrow \cdot cB, B \rightarrow \cdot d \}$$

LR(0) 项目集规范族的构造算法:



- 1. 拓广文法: 在原文法 $G[S]$ 上增加一个产生式 $S' \rightarrow S$, 这是为了得到唯一的接受状态 $S' \rightarrow S \cdot$.
- 2. 设项目集规范族 C 只包含第一个状态 $\{S' \rightarrow \cdot S\}$ 的闭包, 即 $C = \{ \text{Closure}(\{S' \rightarrow \cdot S\}) \}$
- 3. 利用 $G0$ 函数对 C 中的每个项目集和每个符号 X 计算其下一状态, 并将下一状态 $G0(I, X)$ 加入到 C 中, 直到 C 中状态数不再增加
- C 即为文法 G' 的LR(0) 项目集规范族

前述的例子中，是已经拓广了的文法



- | | | |
|------------------------------|-------------------------------|------------------------------|
| 1. $S' \rightarrow \cdot E$ | 2. $S' \rightarrow E \cdot$ | |
| 3. $E \rightarrow \cdot aA$ | 4. $E \rightarrow a \cdot A$ | 5. $E \rightarrow aA \cdot$ |
| 6. $A \rightarrow \cdot cA$ | 7. $A \rightarrow c \cdot A$ | 8. $A \rightarrow cA \cdot$ |
| 9. $A \rightarrow \cdot d$ | 10. $A \rightarrow d \cdot$ | |
| 11. $E \rightarrow \cdot bB$ | 12. $E \rightarrow b \cdot B$ | 13. $E \rightarrow bB \cdot$ |
| 14. $B \rightarrow \cdot cB$ | 15. $B \rightarrow c \cdot B$ | 16. $B \rightarrow cB \cdot$ |
| 17. $B \rightarrow \cdot d$ | 18. $B \rightarrow d \cdot$ | |

初始时，设置 $C = \{ \text{Closure}(\{S' \rightarrow \cdot E\}) \}$ ，是一个状态，
即 $C = \{ \{S' \rightarrow \cdot E, E \rightarrow \cdot aA, E \rightarrow \cdot bB\} \}$
再根据C中的每个状态的go函数，求出更多的状态，直到状态数不再增多为止

例：设文法G为：

$E \rightarrow aA \mid bB$

$A \rightarrow cA \mid d$

$B \rightarrow cB \mid d$

第2步：写出拓广后的文法的项目集：

求该文法的LR(0)分析表。

第1步：拓广文法，
并对产生式给予序号：

(0) $S' \rightarrow E$

(1) $E \rightarrow aA$

(2) $E \rightarrow bB$

(3) $A \rightarrow cA$

(4) $A \rightarrow d$

(5) $B \rightarrow cB$

(6) $B \rightarrow d$

1. $S' \rightarrow \cdot E$

3. $E \rightarrow \cdot aA$

5. $E \rightarrow aA \cdot$

7. $E \rightarrow b \cdot B$

9. $A \rightarrow \cdot cA$

11. $A \rightarrow cA \cdot$

13. $A \rightarrow d \cdot$

15. $B \rightarrow c \cdot B$

17. $B \rightarrow \cdot d$

2. $S' \rightarrow E \cdot$

4. $E \rightarrow a \cdot A$

6. $E \rightarrow \cdot bB$

8. $E \rightarrow bB \cdot$

10. $A \rightarrow c \cdot A$

12. $A \rightarrow \cdot d$

14. $B \rightarrow \cdot cB$

16. $B \rightarrow cB \cdot$

18. $B \rightarrow d \cdot$

第3步：从 $S' \rightarrow \cdot$ 始求项目集规范

由 $S_0 = \{S' \rightarrow \cdot E\}$
知：

$\text{Closure}(S_0) = \{ S' \rightarrow \cdot E,$

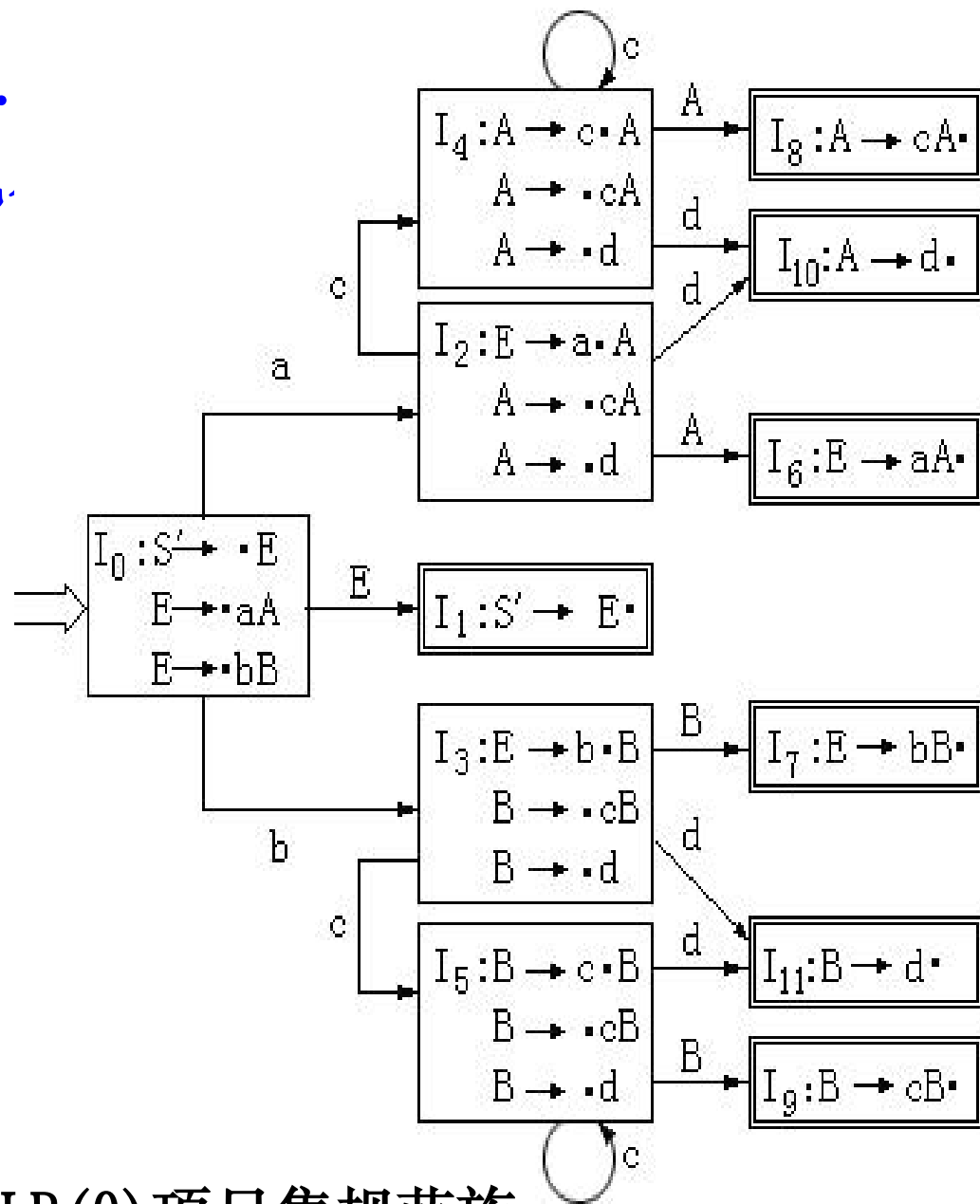
$\rightarrow \cdot E,$

由此初值 $C = \{\text{Closure}(S_0)\}$ 再}
根据GO函数计算后继状态

由此表明明显看出：

$\text{Go}(\text{状态}, \text{后继符号}) =$
后继状态

当不出现新的状态时，得到LR(0)项目集规范族



LR(0) 项目集规范族也可以用表格的形式表示

状态	项目集	后继符号	后继状态
S_0	$\{ S' \rightarrow \cdot E$ $E \rightarrow \cdot aA$ $E \rightarrow \cdot bB \}$	E a b	S_1 S_2 S_3
S_1	$\{ S' \rightarrow E \cdot \}$	#	接受态
S_2	$\{ E \rightarrow a \cdot A$ $A \rightarrow \cdot cA$ $A \rightarrow \cdot d \}$	A c d	S_6 S_4 S_{10}
S_3	$\{ E \rightarrow b \cdot B$ $B \rightarrow \cdot cB$ $B \rightarrow \cdot d \}$	B c d	S_7 S_5 S_{11}
S_4	$\{ A \rightarrow c \cdot A$ $A \rightarrow \cdot cA$ $A \rightarrow \cdot d \}$	A c d	S_8 S_4 S_{10}
S_5	$\{ B \rightarrow c \cdot B$ $B \rightarrow \cdot cB$ $B \rightarrow \cdot d \}$	B c d	S_9 S_5 S_{11}
S_6	$\{ E \rightarrow aA \cdot \}$	#	归约
S_7	$\{ E \rightarrow bB \cdot \}$	#	归约
S_8	$\{ A \rightarrow cA \cdot \}$	#	归约
S_9	$\{ B \rightarrow cB \cdot \}$	#	归约
S_{10}	$\{ A \rightarrow d \cdot \}$	#	归约
S_{11}	$\{ B \rightarrow d \cdot \}$	#	归约

算法描述:

```
PROCEDURE itemsets( $G'$ );  
{  
   $C := \{ \text{CLOSURE}(\{S' \rightarrow \cdot S\}) \};$   
  do {  
    for  $C$  中的每个项目集  $I$  和每个文法符号  $X$   
    do  
      if  $G0(I, X)$  非空且不属于  $C$   
        把  $G0(I, X)$  加入  $C$  中;  
        在  $I$  和  $G0(I, X)$  之间画标记为  $X$  的弧;  
  }while(  $C$  的元素不再增加)  
}
```

4) LR(0) 分析表的构造

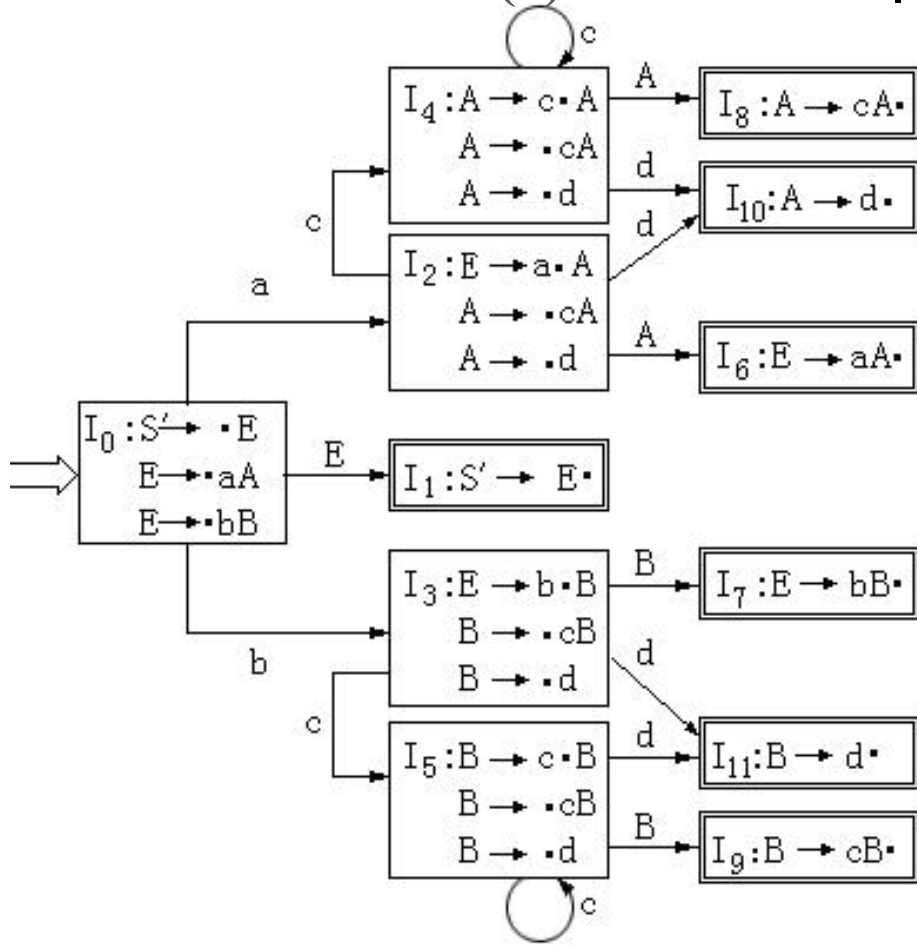
- LR(0) 文法：若一个文法 G 的拓广文法 G' 的活前缀识别自动机中的每个状态不存在下述情况：
 - 既含移进又含归约的项目；
 - 含有多个归约项目。
- 对LR(0)文法可以直接从它的项目集规范族 C 和活前缀识别自动机的状态转换函数构造出LR分析表

设有文法G，则从LR(0)项目集规范族构造分析表的方法为：

- ① 对于 $A \rightarrow \alpha \cdot X\beta \in I_k$, $G0(I_k, X) = I_j$
若 $X \in V_t$, 则置 $action[k, x] = S_j$, 即把 (j, a) 移进栈
若 $X \in V_n$, 则置 $goto[k, X] = j$
- ② 对于 $A \rightarrow \alpha \cdot \in I_k$, 则对所有的 $x \in V_t$ 和 $\#$, 均置 $action[k, x] = r_j$ (设 $A \rightarrow \alpha$ 是第 j 个产生式), 即用 $A \rightarrow \alpha$ 归约
- ③ 若 $S' \rightarrow S \cdot \in I_k$, 则置 $action[k, \#] = acc$, 即接受
- ④ 其他均置出错。

分析表

- 

[illegible]

练习

- 给定文法： $S \rightarrow aS \mid bS \mid c$
 - (1) 构造的LR(0)项目集规范族，
 - (2) 构造识别该文法所产生的活前缀的DFA
 - (3) 该文法是否是LR(0)的，若是，构造LR(0)分析表

一. 拓广文法

二. 写出所有的项目

(0) $S' \rightarrow S$

(1) $S \rightarrow aS$

(2) $S \rightarrow bS$

(3) $S \rightarrow c$

1. $S' \rightarrow \cdot S$ 2. $S' \rightarrow S \cdot$

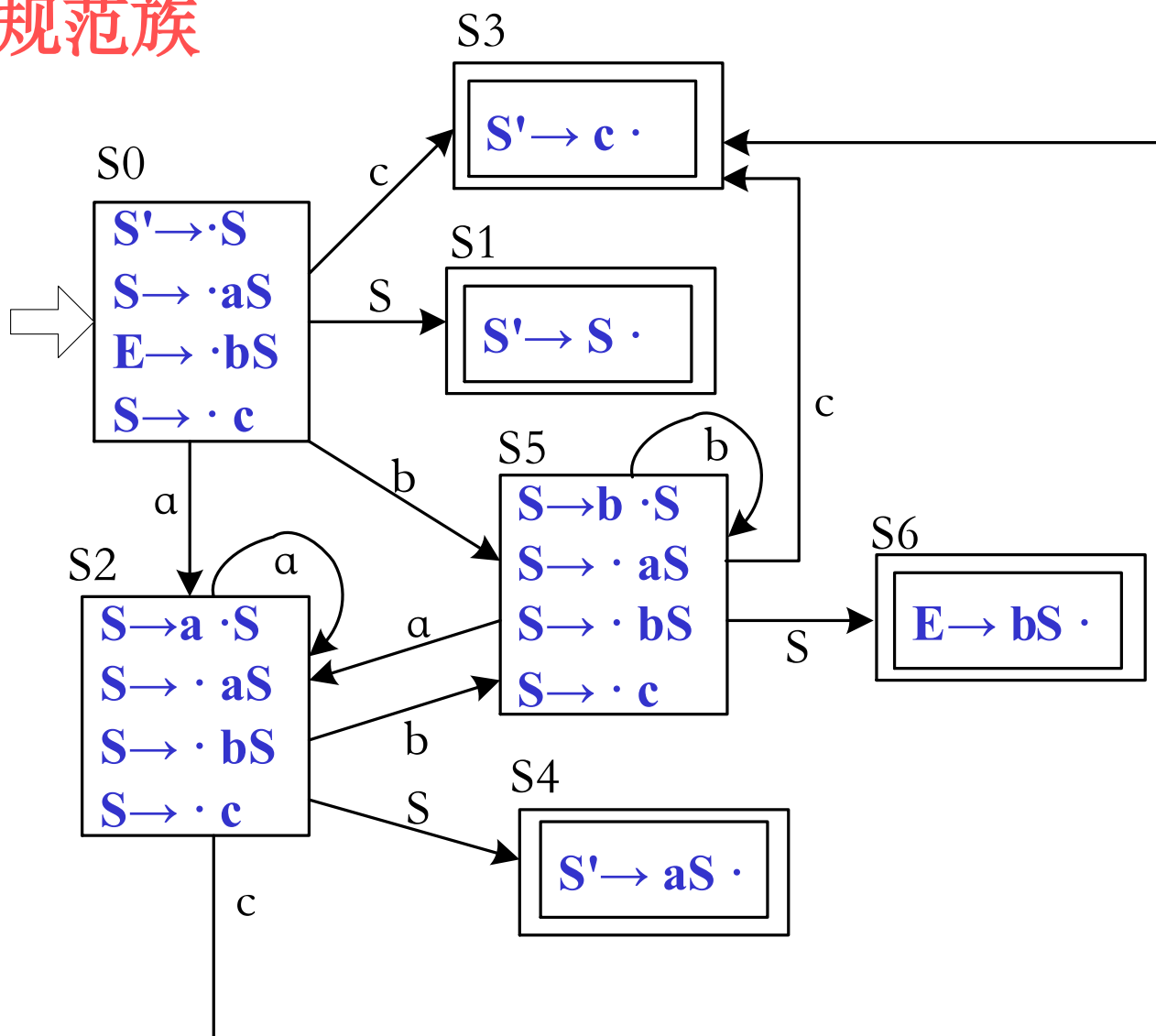
3. $S \rightarrow \cdot aS$ 4. $S \rightarrow a \cdot S$ 5. $S \rightarrow aS \cdot$

6. $S \rightarrow \cdot bS$ 7. $S \rightarrow b \cdot S$ 8. $S \rightarrow bS \cdot$

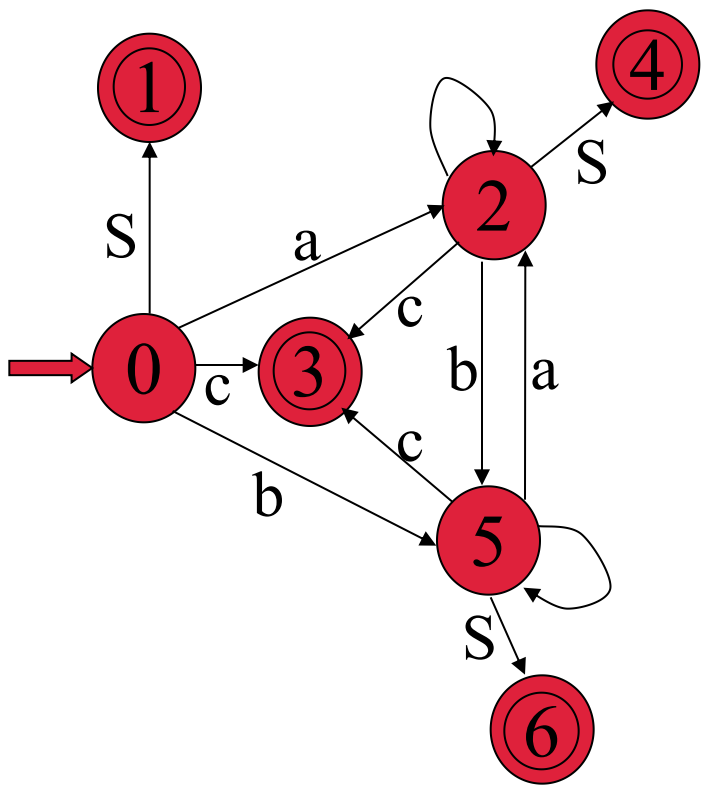
9. $S \rightarrow \cdot c$ 10. $S \rightarrow c \cdot$

三. 求项目集规范族

(0) $S' \rightarrow S$
 (1) $S \rightarrow aS$
 (2) $S \rightarrow bS$
 (3) $S \rightarrow c$



项目集规范族也可以用表格表示



状态	项目集	后继符号	后继状态
S_0	$\{ S' \rightarrow \cdot S$ $S \rightarrow \cdot aS$ $S \rightarrow \cdot bS$ $S \rightarrow \cdot c \}$	S a b c	S_1 S_2 S_5 S_3
S_1	$\{ S' \rightarrow S \cdot \}$	#	接受态
S_2	$\{ S \rightarrow a \cdot S$ $S \rightarrow \cdot aS$ $S \rightarrow \cdot bS$ $S \rightarrow \cdot c \}$	S a b c	S_4 S_2 S_5 S_3

每个项目集中的各个项目不冲突,则是LR(0)文法。

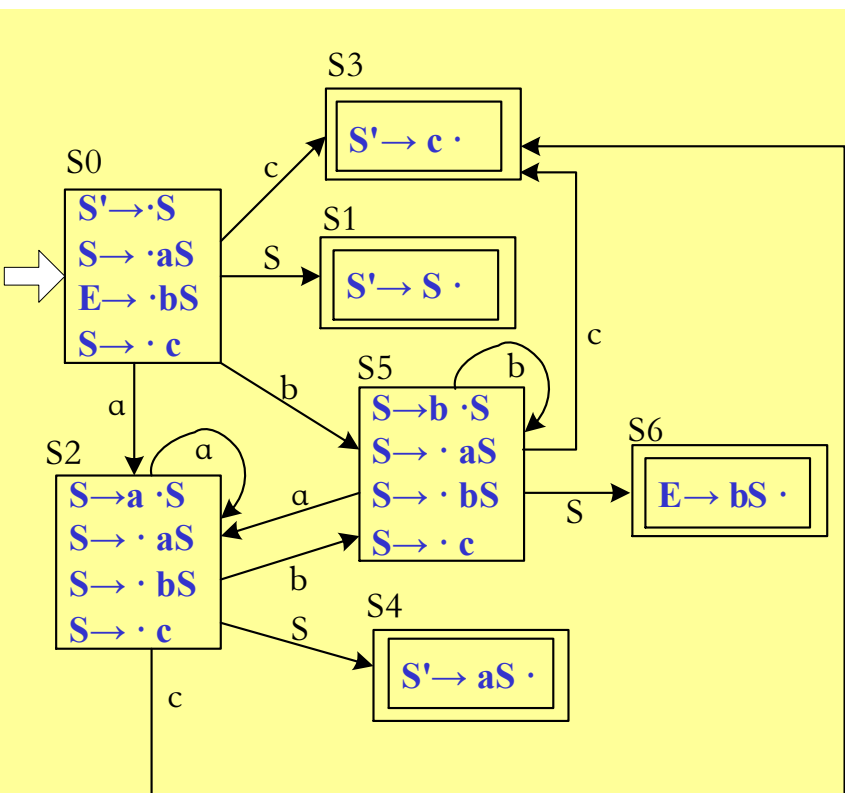
四. 构造LR(0)分析表

(0) $S' \rightarrow S$

(1) $S \rightarrow aS$

(2) $S \rightarrow bS$

(3) $S \rightarrow c$

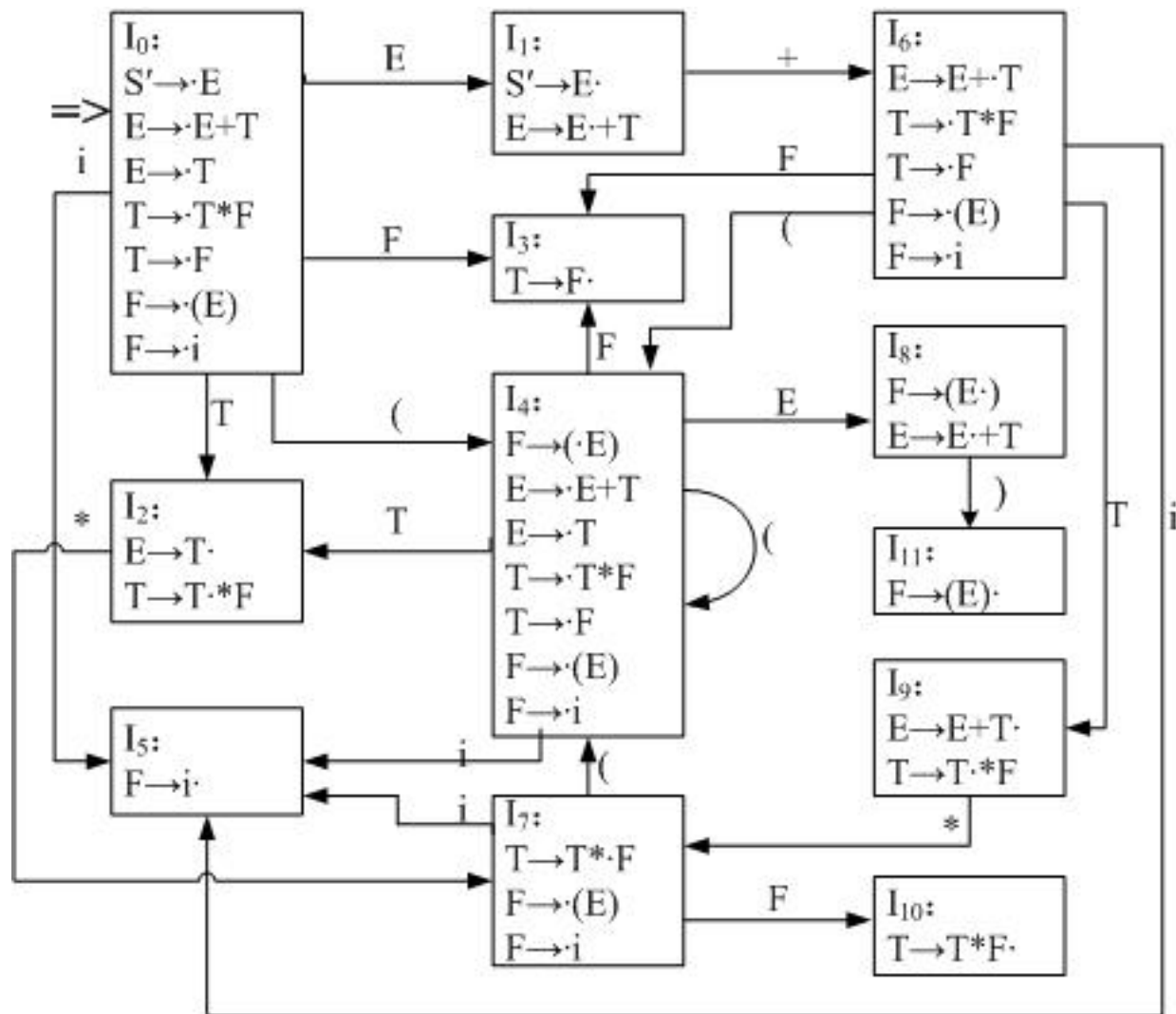


状态	ACTION				GOTO
	a	b	c	#	
0	S_2	S_5	S_3		1
1				acc	
2	S_2	S_5	S_3		4
3	r_3	r_3	r_3	r_3	
4	r_1	r_1	r_1	r_1	
5	S_2	S_5	S_3		6
6	r_2	r_2	r_2	r_2	

非LR(0) 文法

考查文法：

- (0) $S' \rightarrow E$
- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow i$



- 项目集规范族中 I_1 , I_2 , I_9 中存在移进-归约冲突，不能构造LR(0)分析表

3 SLR分析方法

- 只有当一个文法G是LR(0)文法，即G的每一个状态不出现冲突时，才能构造出LR(0)分析表。
- 由于大多数实用的程序设计语言的文法不能满足LR(0)文法的条件，因此使用向前查看一个符号的办法来处理冲突。
- 只对有冲突的状态简单向前查看一个符号，即查看follow集，以确定做哪个动作，这种分析方法为简单的LR分析法，用SLR表示。
- 如果每个项目都附上k个终结符号，表示要对它进行归约时，后续的k个输入符号与这k个符号相同时，才能对栈顶进行归约。这就是LR(k)分析。

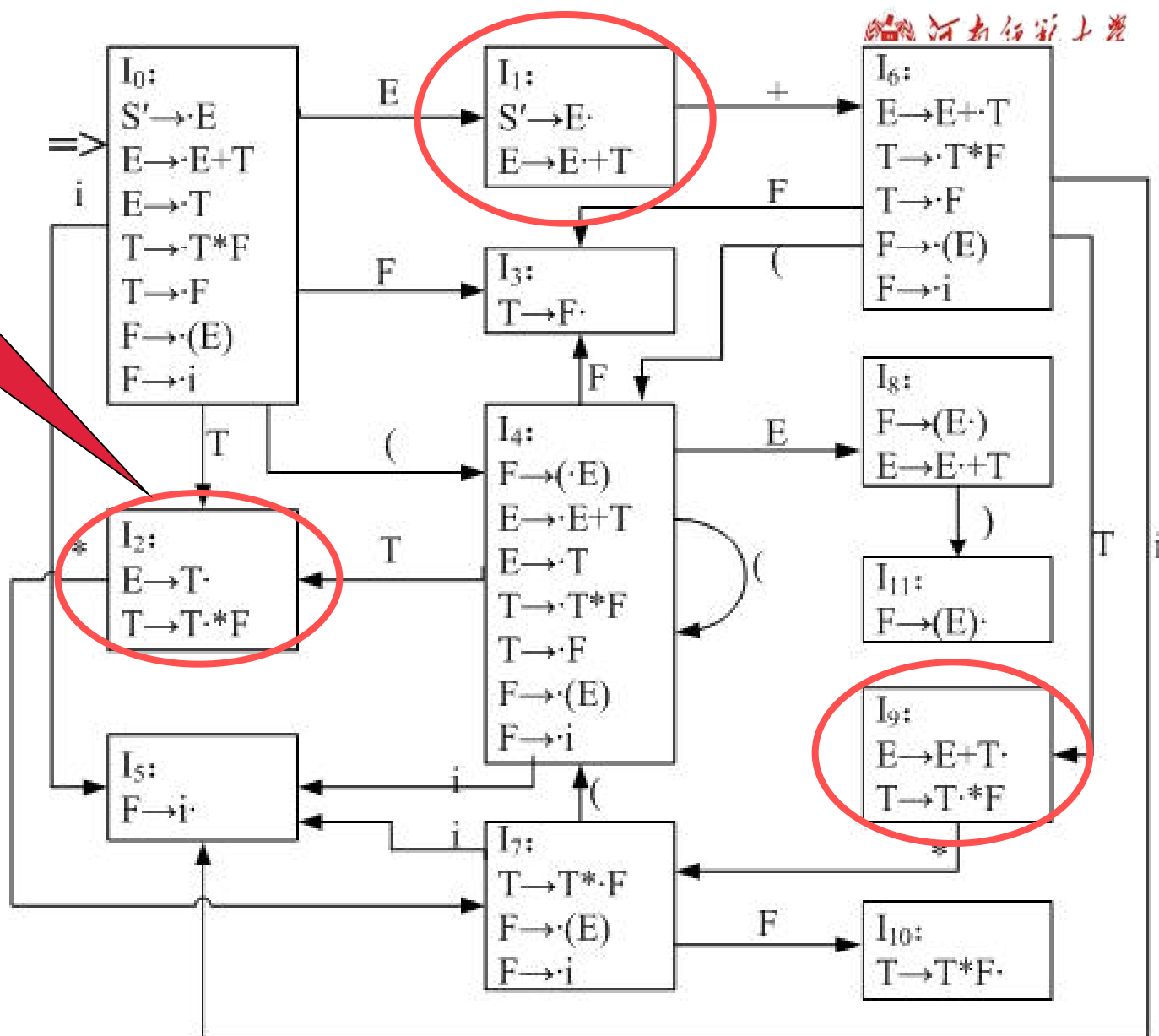
SLR(1) 冲突的解决办法

$$I = \left\{ \begin{array}{l} X \rightarrow \alpha \cdot b A \\ A \rightarrow \beta \cdot \\ B \rightarrow \gamma \cdot \end{array} \right\}$$

- 若一个项目集中同时含有移进和归约项目，出现了冲突。
- **解决冲突的条件**：移进符号集合 $\{b\}$, $\text{Follow}(A)$, $\text{follow}(B)$ 两两不相交。
- **解决的办法**：当面临的输入符号为 a ：
 - 当 $a \in \text{移进符号 } b$ ，则应移进；
 - 当 $a \in \text{follow}(A)$ ，则用产生式 $A \rightarrow \beta$ 进行归约；
 - 当 $a \in \text{follow}(B)$ ，则用产生式 $B \rightarrow \gamma$ 进行归约。

移进-归约冲突

- (0) $S' \rightarrow E$
- (1) $E \rightarrow E+T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow i$



冲突的解决方法：

- 解决冲突的条件：

在 I_1 中， $\text{Follow}(S') = \{\#\}$ ，
 $\{\#\} \cap \{+\} = \Phi$ ，因此 I_1 中的冲突可解决。

$S' \rightarrow E \cdot$

$E \rightarrow E \cdot + T$

- 解决方法：

当且仅当遇到句子的结束符“#”号时，使用 $S' \rightarrow E$ 进行归约，句子被接受；当遇到“+”时，移进；当面临其他输入符号时，出错。

- 解决条件:

在 I_2 中, $\text{Follow}(E) = \{\#,), +\}$

$$\text{Follow}(E) \cap \{*\} = \{+,), \#\} \cap \{*\} = \Phi$$

所以 I_2 的冲突可以解决

$E \rightarrow T \cdot$

$T \rightarrow T \cdot * F$

- 解决方法:

如果当前输入符号为 $\#,), +$ 时,
就使用 $E \rightarrow T$ 进行归约; 如果当
前输入符号为 $*$ 时, 就移进; 当
面临其他输入符号时, 出错。

$E \rightarrow E + T \cdot$
 $T \rightarrow T \cdot * F$

- 解决条件：
- 在 I_9 中，
 $\text{Follow}(E) \cap \{*\} = \{+,), \#\} \cap \{*\} = \Phi$ ，因此 I_9 中的冲突可解决
- 解决方法：
- 面临的输入符为 ‘+’， ‘)’
或 ‘#’ 时，用产生式 $E \rightarrow E + T$
进行归约；当面临输入符为
‘*’ 时，移进；面临其它符
号则报错。

SLR(1)分析表的构造算法

设有文法 G ，则SLR(1)分析表的构造方法为：

- ① 对于 $A \rightarrow a \cdot X\beta \in I_k$ ， $G0(I_k, X) = I_j$
若 $X \in V_t$ ，则置 $action[k, X] = S_j$ ，即把 (j, a) 移进栈
若 $X \in V_n$ ，则置 $goto[k, X] = j$
- ② 对于 $A \rightarrow a \cdot \in I_k$ ，则对所有的 x ， $x \in follow(A)$ ，
则置 $action[k, x] = r_j$ （设 $A \rightarrow a$ 是第 j 个产生式），即用 $A \rightarrow a$ 归约
- ③ 若 $S' \rightarrow S \cdot \in I_k$ ，则置 $action[k, \#] = acc$ ，即接受
- ④ 其他均置出错。

如果分析表的每个入口不含多重定义，则称为SLR表，文法 G 称为SLR(1)文法。



例：下述文法的SLR分析表

- (0) $S' \rightarrow E$
- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow i$

状态	ACTION						GoTo		
	i	+	*	(	)	#	E	T	F
0	S ₅			S ₄			1	2	3
1		S ₆				acc			
2		r ₂	S ₇		r ₂	r ₂			
3		r ₄	r ₄		r ₄	r ₄			
4	S ₅			S ₄			8	2	3
5		r ₆	r ₆		r ₆	r ₆			
6	S ₅			S ₄		r ₁		9	3
7	S ₅			S ₄					10
8		S ₆			S ₁₁				
9		r ₁	S ₇		r ₁	r ₁			
10		r ₃	r ₃		r ₃	r ₃			
11		r ₅	r ₅		r ₅	r ₅			

小结

- 1. 拓广文法：在原文法 $G[S]$ 上增加一个产生式 $S' \rightarrow S$ ，这是为了得到唯一的接受状态 $S' \rightarrow S \cdot$ 。
- 2. 设项目集规范族 C 只包含第一个状态 $\{S' \rightarrow \cdot S\}$ 的闭包，即 $C = \{ \text{Closure}(\{S' \rightarrow \cdot S\}) \}$
- 3. 利用 $G0$ 函数对 C 中的每个项目集和每个符号 X 计算其下一状态，并将下一状态 $G0(I, X)$ 加入到 C 中，直到 C 中状态数不再增加
- C 即为文法 G' 的 $LR(0)$ 项目集规范族

小结

设有文法 G ，则从 $LR(0)$ 项目集规范族构造分析表的方法为：

- ① 对于 $A \rightarrow a \cdot X\beta \in I_k$ ， $G0(I_k, X) = I_j$
若 $X \in V_t$ ，则置 $action[k, x] = S_j$ ，即把 (j, a) 移进栈
若 $X \in V_n$ ，则置 $goto[k, X] = j$
- ② 对于 $A \rightarrow a \cdot \in I_k$ ，则对所有的 $x \in V_t$ 和 $\#$ ，均置 $action[k, x] = r_j$ （设 $A \rightarrow a$ 是第 j 个产生式），即用 $A \rightarrow a$ 归约
- ③ 若 $S' \rightarrow S \cdot \in I_k$ ，则置 $action[k, \#] = acc$ ，即接受
- ④ 其他均置出错。